

Лабораторная работа 11. Модель системы массового обслуживания M | M | 1

Абакумова Олеся Максимовна, НФИбд-02-22

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Построение модели с помощью CPNTools	8
4.2	Мониторинг параметров моделируемой системы	13
5	Выводы	20

Список иллюстраций

4.1	Граф сети системы обработки заявок в очереди	8
4.2	Граф генератора заявок системы	10
4.3	Параметры элементов генератора заявок системы	11
4.4	Параметры элементов обработчика заявок системы	12
4.5	Запуск модели	12
4.6	Настройка мониторинга	13
4.7	Запуск системы обработки заявок в очереди	14
4.8	Сгенерированный файл	15
4.9	Функция Observer	17
4.10	Задани longdelaytime	17
4.11	Содержание Queue_Delay.log	18
4.12	Периоды времени, когда значения задержки в очереди превышали заданное значение	19

Список таблиц

1 Цель работы

Реализовать модель задачи системы массового обслуживания $M|M|1$

2 Задание

1. Построить модель задачи системы массового обслуживания $M|M|1$
2. Построить график

3 Теоретическое введение

В систему поступает поток заявок двух типов, распределённый по пуассоновскому закону. Заявки поступают в очередь сервера на обработку. Дисциплина очереди - FIFO. Если сервер находится в режиме ожидания (нет заявок на сервере), то заявка поступает на обработку сервером.

4 Выполнение лабораторной работы

4.1 Построение модели с помощью CPNTools

Будем использовать три отдельных листа: на первом листе опишем граф системы, на втором — генератор заявок, на третьем — сервер обработки заявок. Сеть имеет 2 позиции (очередь — Queue, обслуженные заявки — Completed) и два перехода (генерировать заявку — Arrivals, передать заявку на обработку серверу — Server). Переходы имеют сложную иерархическую структуру, задаваемую на отдельных листах модели (с помощью соответствующего инструмента меню — Hierarchy) (рис. 4.1):

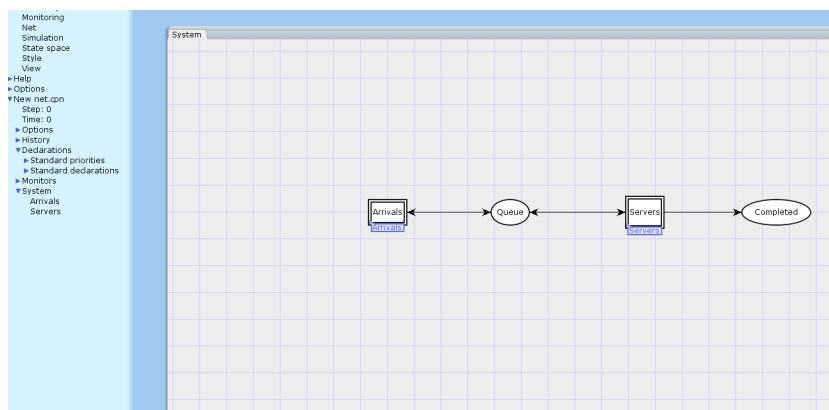


Рис. 4.1: Граф сети системы обработки заявок в очереди

Зададим декларации системы. Определим множества цветов системы (colorset): - фишки типа UNIT определяют моменты времени; - фишки типа INT определяют моменты поступления заявок в систему. - фишки типа JobType определяют 2 типа заявок — А и В; - кортеж Job имеет 2 поля: jobType опреде-

ляет тип работы (соответственно имеет тип JobType, поле AT имеет тип INT и используется для хранения времени нахождения заявки в системе; - фишки Jobs — список заявок; - фишки типа ServerxJob — определяют состояние сервера, занятого обработкой заявок.

```
colset UNIT = unit timed;
colset INT = int;
colset Server = with server timed;
colset JobType = with A | B;
colset Job = record jobType : JobType *
AT : INT;
colset Jobs = list Job;
colset ServerxJob = product Server * Job timed;
```

Переменные модели: - proctime — определяет время обработки заявки; - job — определяет тип заявки; - jobs — определяет поступление заявок в очередь. var proctime : INT; var job: Job; var jobs: Jobs; Определим функции системы: - функция expTime описывает генерацию целочисленных значений через интервалы времени, распределённые по экспоненциальному закону; - функция intTime преобразует текущее модельное время в целое число; - функция newJob возвращает значение из набора Job — случайный выбор типа заявки (A или B).

```
fun expTime (mean: int) =
let
val realMean = Real.fromInt mean
val rv = exponential((1.0/realMean))
in
floor (rv+0.5)
end;
fun intTime() = IntInf.toInt (time());
```

```
fun newJob() = {jobType = JobType.ran(),
AT = intTime()}
```

Между переходом Arrivals и позицией Queue, а также между позицией Queue и переходом Server установлена дуплексная связь. Между переходом Server и позицией Completed — односторонняя связь. Граф генератора заявок имеет 3 позиции (текущая заявка — Init, следующая заявка — Next, очередь — Queue из листа System) и 2 перехода (Init — определяет распределение поступления заявок по экспоненциальному закону с интенсивностью 100 заявок в единицу времени, Arrive — определяет поступление заявок в очередь) На листе System (рис. 11.4):

- у позиции Queue множество цветов фишек — Jobs; начальная маркировка 1'[] определяет, что изначально очередь пуста.
- у позиции Completed множество цветов фишек — Job.(рис. 4.2):

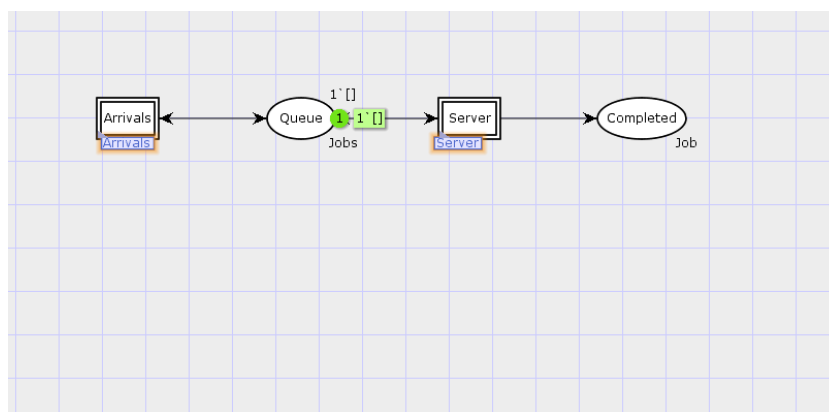


Рис. 4.2: Граф генератора заявок системы

На листе Arrivals (рис. 11.5):

- у позиции Init: множество цветов фишек — UNIT; начальная маркировка 1'()[0?] определяет, что поступление заявок в систему начинается с нулевого момента времени;
- у позиции Next: множество цветов фишек — UNIT;
- на дуге от позиции Init к переходу Init выражение () задаёт генерацию заявок;
- на дуге от переходов Init и Arrive к позиции Next выражение ()@+expTime(100) задаёт экспоненциальное распределение времени между поступлениями заявок;
- на дуге от позиции Next к переходу Arrive выражение ()

задаёт перемещение фишки; - на дуге от перехода Arrive к позиции Queue выражение $jobs^{1^1}$ задаёт поступление заявки в очередь; - на дуге от позиции Queue к переходу Arrive выражение $jobs$ задаёт обратную связь.

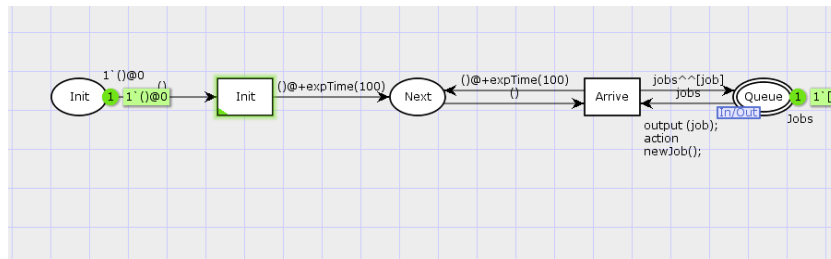


Рис. 4.3: Параметры элементов генератора заявок системы

Граф процесса обработки заявок на сервере имеет 4 позиции (Busy — сервер занят, Idle — сервер в режиме ожидания, Queue и Completed из листа System) и 2 перехода (Start — начать обработку заявки, Stop — закончить обработку заявки) На листе Server: - у позиции Busy: множество цветов фишек — Server, начальное значение маркировки — $1'server@0$ определяет, что изначально на сервере нет заявок на обслуживание; - у позиции Idle: множество цветов фишек — $ServerxJob$; - переход Start имеет сегмент кода `output (proctime); action expTime(90);` определяющий, что время обслуживания заявки распределено по экспоненциальному закону со средним временем обработки в 90 единиц времени; - на дуге от позиции Queue к переходу Start выражение $job::jobs$ определяет, что сервер может начать обработку заявки, если в очереди есть хотя бы одна заявка; - на дуге от перехода Start к позиции Busy выражение $(server,job)@+proctime$ запускает функцию расчёта времени обработки заявки на сервере; - на дуге от позиции Busy к переходу Stop выражение $(server,job)$ говорит о завершении обработки заявки на сервере; - на дуге от перехода Stop к позиции Completed выражение job показывает, что заявка считается обслуженной; - выражение $server$ на дугах от и к позиции Idle определяет изменение состояние сервера (обрабатывает заявки или ожидает); - на дуге от перехода Start к позиции Queue выражение $jobs$ задаёт обратную связь.(рис. 4.4):

¹job

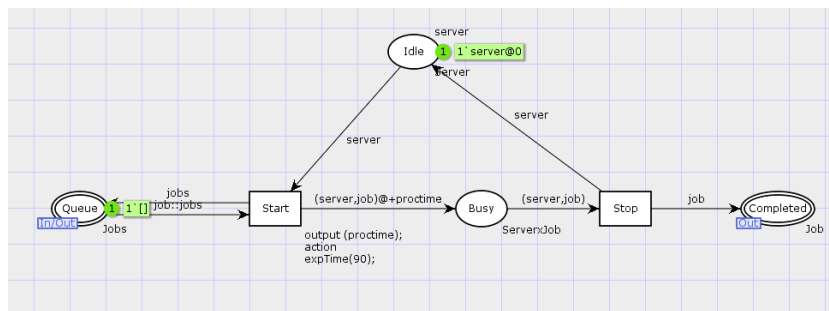


Рис. 4.4: Параметры элементов обработчика заявок системы

```

#!/usr/bin/gnuplot -persist
# задаём текстовую кодировку,
# тип терминала, тип и размер шрифта
set encoding utf8
set term pngcairo font "Helvetica,9"
# задаём выходной файл графика
set out 'window_1.png'
set style line 2
plot [0:] [0:1.2] "Long_Delay_Time.log" using ($4):($1) with lines

```

Запустив модель мы можем наблюдать как один лист сменяет другой в ходе работы (рис. 4.5):

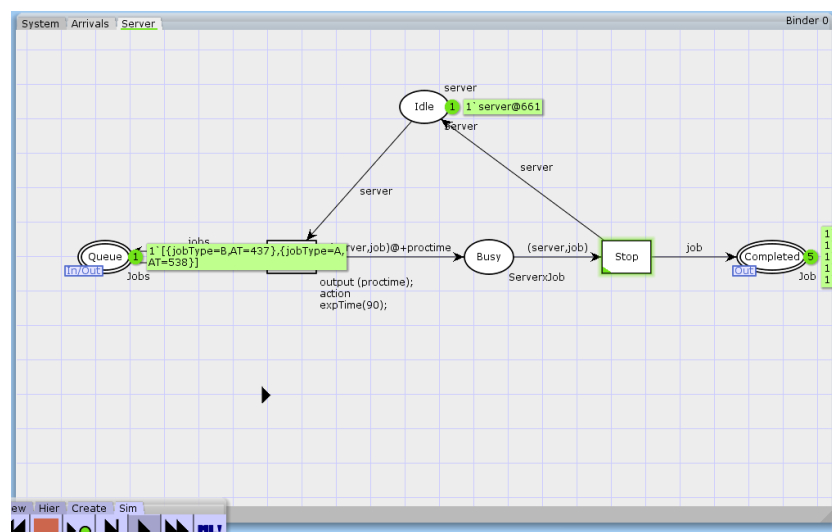


Рис. 4.5: Запуск модели

4.2 Мониторинг параметров моделируемой системы

Потребуется палитра Monitoring. Выбираем Break Point (точка останова) и устанавливаем её на переход Start. После этого в разделе меню Monitor появится новый подраздел, который назовём Ostanovka. В этом подразделе необходимо внести изменения в функцию Predicate, которая будет выполняться при запуске монитора. Изначально, когда функция начинает работать, она возвращает значение true, в противном случае — false. В теле функции вызывается процедура predBindElem, которую определяем в предварительных декларациях. Зададим число шагов, через которое будем останавливать мониторинг. Для этого true заменим на Queue_Delay.count()=200. Необходимо определить конструкцию Queue_Delay.count(). С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Появившийся в меню монитор называем Queue Delay (без подчеркивания) (рис. 4.6):

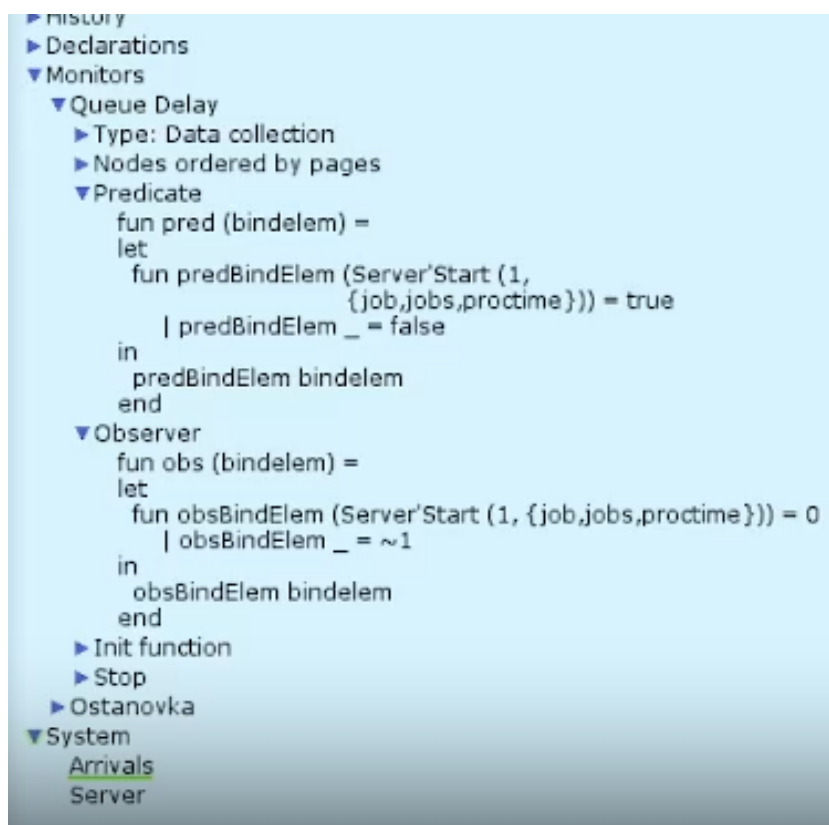


Рис. 4.6: Настройка мониторинга

При запуске мы получаем следующее (рис. 4.7):

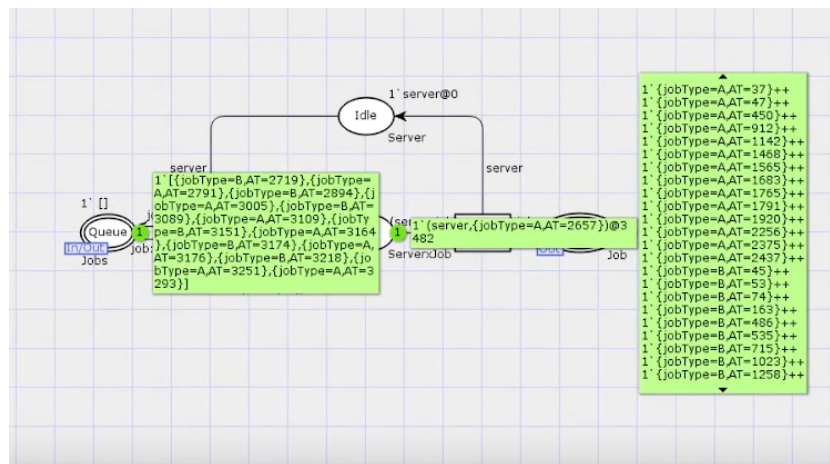
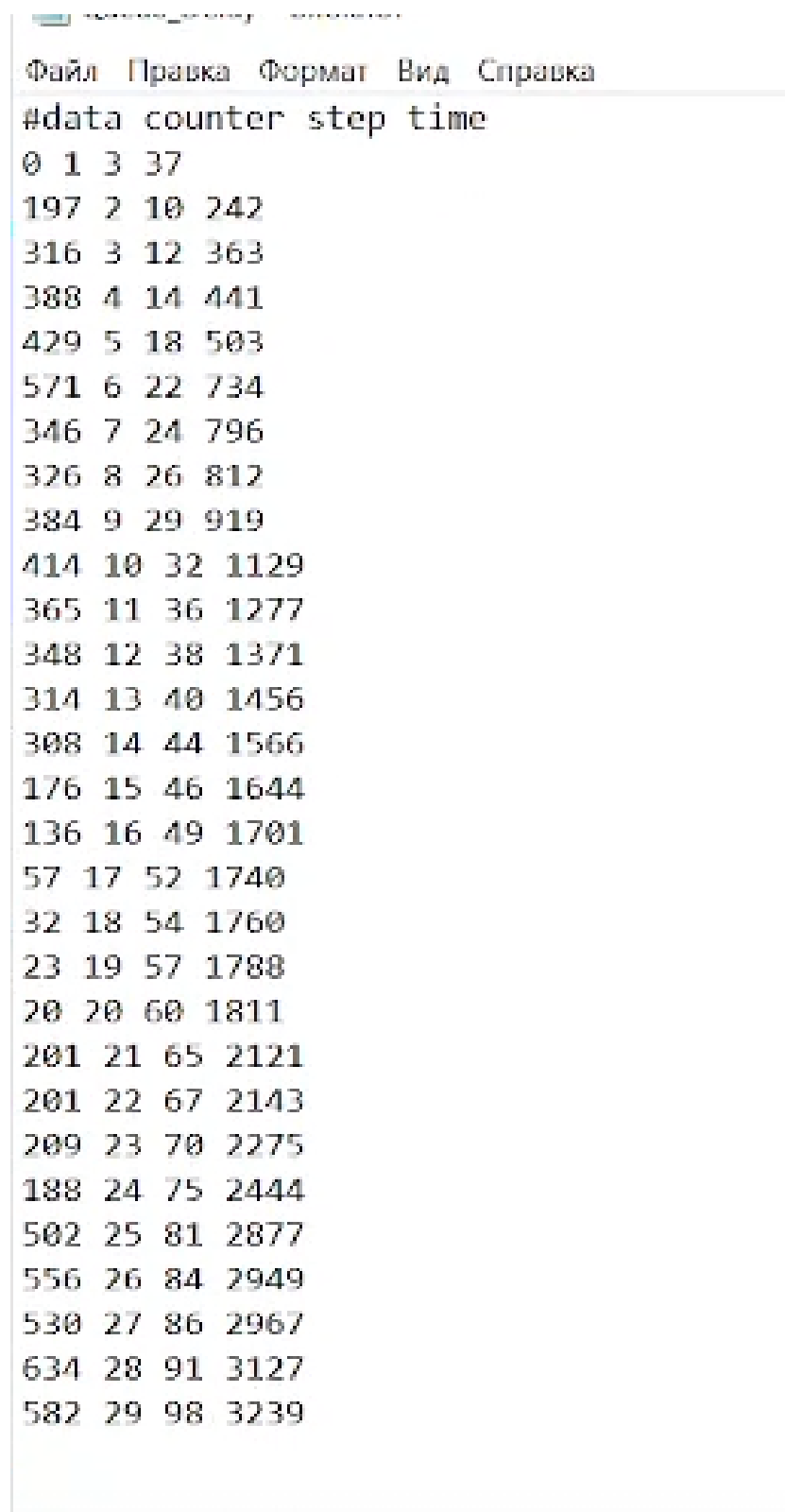


Рис. 4.7: Запуск системы обработки заявок в очереди

После запуска получаем следующий файл в каталоге (рис. 4.8):



The image shows a screenshot of a text editor window. The title bar at the top reads "Untitled - Notepad++". The menu bar includes "Файл", "Правка", "Формат", "Вид", and "Справка". The text content is a data file with a header line "#data counter step time" and 30 subsequent lines of numerical data. The data is organized into four columns: a primary value, a counter, a step, and a time value. The primary values decrease from 197 to 582, while the other three values increase incrementally.

```
Файл Правка Формат Вид Справка
#data counter step time
0 1 3 37
197 2 10 242
316 3 12 363
388 4 14 441
429 5 18 503
571 6 22 734
346 7 24 796
326 8 26 812
384 9 29 919
414 10 32 1129
365 11 36 1277
348 12 38 1371
314 13 40 1456
308 14 44 1566
176 15 46 1644
136 16 49 1701
57 17 52 1740
32 18 54 1760
23 19 57 1788
20 20 60 1811
201 21 65 2121
201 22 67 2143
209 23 70 2275
188 24 75 2444
502 25 81 2877
556 26 84 2949
530 27 86 2967
634 28 91 3127
582 29 98 3239
```

Рис. 4.8: Сгенерированный файл

Функция Observer выполняется тогда, когда функция предикатора выдаёт значение true. По умолчанию функция выдаёт 0 или унарный минус (~1), подчёркивание обозначает произвольный аргумент:

```
fun obs (bindelem) =  
  let  
    fun obsBindElem (Server'Start (1, {job, jobs, proctime})) = 0  
    | obsBindElem _ = ~1  
  in  
    obsBindElem bindelem  
  end
```

Изменим её так, чтобы получить значение задержки в очереди. Для этого необходимо из текущего времени intTime() вычесть временную метку AT , означающую приход заявки в очередь(рис. 4.9):

```
un obs (bindelem) =  
  let  
    fun obsBindElem (Server'Start (1, {job, jobs, proctime}))  
    = (intTime() - (#AT job))  
    | obsBindElem _ = ~1  
  in  
    obsBindElem bindelem  
  end
```

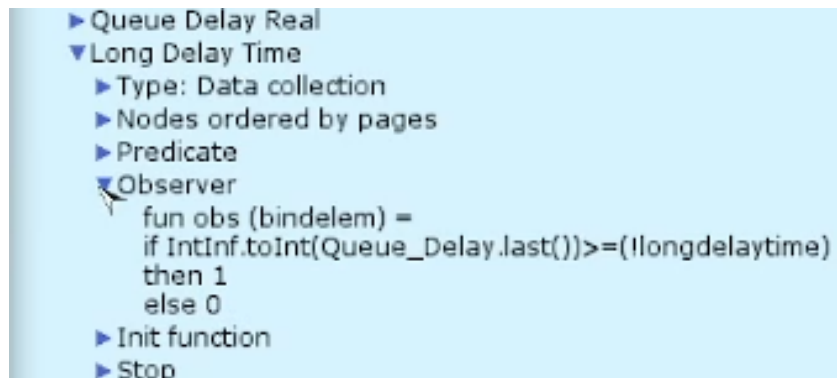



Рис. 4.9: Функция Observer

Важно также в задать параметр longdelaytime (рис. 4.10):

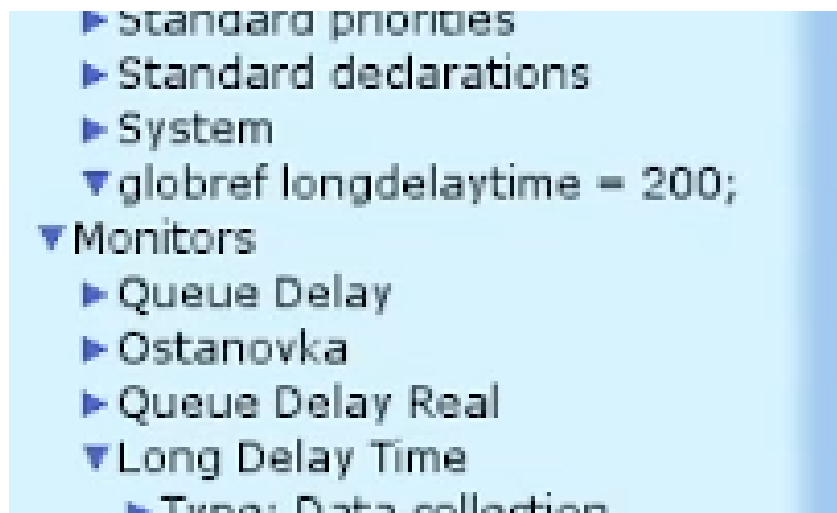


Рис. 4.10: Задани longdelaytime

После запуска программы на выполнение в каталоге с кодом программы появится файл Queue_Delay.log, содержащий в первой колонке — значение задержки очереди, во второй — счётчик, в третьей — шаг, в четвёртой — время (рис. 4.11):

```

#data counter step time
0 1 3 79
0 2 7 220
0 3 11 304
0 4 15 416
1 5 20 585
1 6 22 600
1 7 24 698
1 8 26 701
1 9 28 805
1 10 31 866
0 11 33 919
0 12 36 1076
0 13 39 1098
0 14 43 1202
0 15 45 1291
0 16 48 1482
0 17 51 1549
0 18 54 1799
0 19 59 2011
0 20 61 2135
0 21 64 2189
0 22 66 2208
0 23 69 2291
0 24 72 2530
0 25 75 2588
0 26 78 2636
0 27 81 2805
1 28 85 3101
0 29 87 3102
1 30 92 3401
1 31 94 3540
1 32 96 3560

```

Рис. 4.11: Содержание Queue_Delay.log

С помощью gnuplot можно построить график значений задержки в очереди, выбрав по оси x время, а по оси y — значения задержки (рис. 4.12):

```
#!/usr/bin/gnuplot -persist
# задаём текстовую кодировку,
# тип терминала, тип и размер шрифта
set encoding utf8
set term pngcairo font "Helvetica,9"
# задаём выходной файл графика
set out 'window_1.png'
set style line 2
plot [0:] [0:1.2] "Long_Delay_Time.log" using ($4):($1) with lines
```

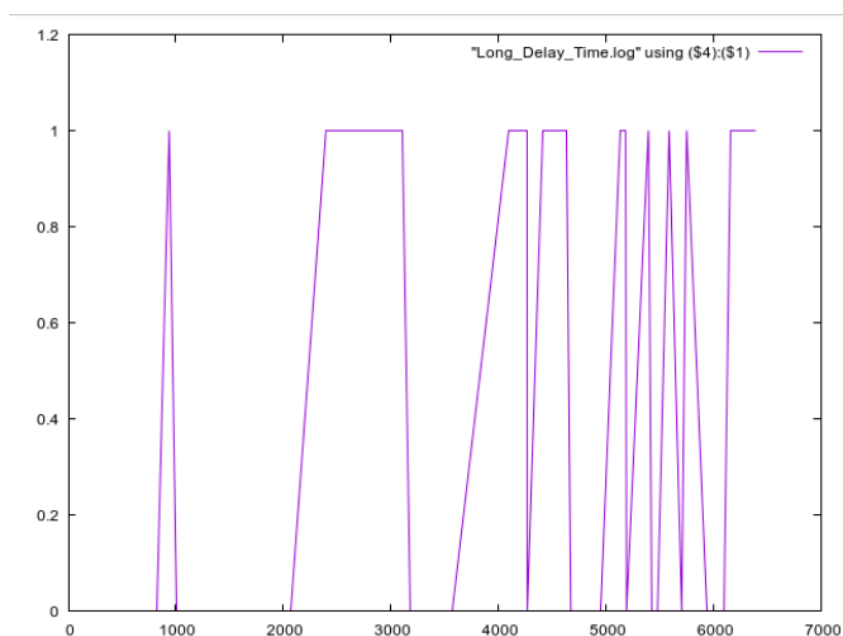


Рис. 4.12: Периоды времени, когда значения задержки в очереди превышали заданное значение

5 Выводы

В процессе выполнения данной лабораторной работы я реализовала модель системы массового обслуживания $M|M|1$ в CPN Tools.